instructBrain

The six changes, and exactly what each one does

Report integrity: what changes, what you will notice, and what needs the database

The whole thing in five lines

instructBrain builds and publishes reports well. The gap is **proving what was sent, and that a person approved it**.

Today a share link shows whatever the report says *right now*, and allocating a trade silently counts as confirming the wording. Both are fixable in a day.

Four of the six changes are code only. **One needs a database change**, which is why nothing here has been applied yet.

Nothing in this document has been built. Nothing is committed. Nothing has run against the live database.

Prepared for Darren Hopwood | **Prepared by** Banksy, instructBrain agent account

Date 11 October 2026 | **Product** instructBrain (instructbrain.com) | **Status** proposal - not applied

Every file reference and line number in this document was read at commit `1d47100` on 11 October 2026, which is the tip of `main`. The findings are the ones Codex raised and I re-checked at source; where I disagree with Codex, it says so.

SECTION 1

The six changes at a glance

Five findings came back from the review. Two are real launch blockers, one is a serious defect, two are design debt. Here is every change, in the order I would do them.

#	Change	Type	Needs DB?	Size
1	**Share links stop showing live content**	Launch blocker	Yes - one column	Small
2	**Confirming and allocating stop being the same thing**	Launch blocker	No	Small
3	**The publish gate moves onto the server**	Serious defect	Yes - a function	Medium
4	**Nothing external exists for an unpublished report**	Serious defect	No	Small
5	**The close-out email link points somewhere a supplier can open**	Broken journey	No	One line
6	**The reason for a recipient gets recorded**	Design debt	Yes - a column	Small

What you will actually notice as a user

Today	After
You reopen a published report to fix a typo, and a client holding last week's link sees your half-edited draft.	The client keeps seeing the version you sent. Your edits wait for the next version.
You assign "Bricklayer" to an item. It silently counts as you having read and approved the wording.	Assigning a trade does only that. Approving wording stays its own act, and an edit takes the approval back off.
Publish is blocked in the browser, but nothing stops the same action being sent straight to the database.	The database refuses it. The gate is no longer a suggestion.
A client email sent too early arrives with no link and no attachment - an empty envelope.	The send is refused, and says why. Nothing half-sent.
The close-out email tells a subcontractor to visit a page they cannot open without a login.	It links to their own trade page, which needs no account.

> **The honest bit**
>
> Changes 1 and 2 are the ones that matter. If you only ever did those two, the product would stop making promises it cannot keep. Changes 3 to 6 are tidying and hardening - worth doing, not worth delaying a launch for.

SECTION 2 - THE TWO THAT MATTER

Change 1: share links stop showing live content

What is wrong now. Publishing does freeze a copy of the report - that part is done properly. But the public link never reads it. It reads the live tables, so the link always shows the current state of the report, not the state you sent.

Today

The public page loads `reports`, `findings` and `photos` live (`shared-report.$token.ts:61-88`). Only three things in the whole codebase read the frozen copy - the emailed PDF attachment, the version list, and the insert.

The share table has no version column at all (`report_shares`, migration `20260806181751`), so a link cannot be tied to one.

After

Minting a link records which version it points at. The public page serves that frozen copy.

A link can no longer drift, because it is no longer reading anything that can change.

The sharper half, which the review missed and I found

`reopenReport` (`report-data.ts:492-501`) sets the report back to `in_review`. The public page only checks status on the manual-report branch (`shared-report.$token.ts:93-101`).

So the moment you reopen a published report to fix one line, **every link already in a client's inbox starts serving your working copy**. Not stale evidence - unpublished draft content, presented as final, with nothing on the page to say so.

Exactly what I change

- Add `version_id` to `report_shares`, and set it when a link is minted (`createShareLink`, `report-data.ts:533-563`).
- Serve the public page from that frozen version when it is set.
- **Refuse to serve live content at all whenever the report is not published** - for every report, not only the manual ones. This is the same rule already applied to the manual branch, applied to the other branch of the same page.
- Old links with no version recorded: bind them to the report's current version on read, so they stop drifting too.

What this costs you

During the window where you have a published report reopened for editing, a client clicking their link now sees a short "this report is not currently published" message instead of your draft. That is the correct trade - but it is a visible behaviour change, so I am telling you now rather than after.

Change 2: confirming and allocating stop being the same thing

What is wrong now. There are two ideas here - "I have read this wording and I stand behind it" and "this is a bricklayer's job". The code treats them as one act.

What you do	What the code writes	Where
Edit the wording of a finding	Writes the text. **Leaves the existing approval in place**	`reports.$id.index.tsx:180-194`
Assign a trade to one item	Writes `confirmed_at` and `confirmed_by` - records allocation as approval	`:216-217`
Assign a trade to many items	Same, once per item	`:253-254`
"Use all suggestions" in the organiser	Applies the below-threshold suggestions too, and they land on the same path	`trade-organiser.tsx:130-152`

Why this is a launch blocker, not tidying

Publishing is blocked on three things: an item not assessed, a failing item with no trade, and an item not confirmed (`document.ts:252-275`).

Allocating a trade clears two of the three at once, and Publish then lights up (`report-actions.tsx:512`). So a report can be issued **asserting that every item was read and approved by a person, when nobody read the wording.**

On your own test data, read-only: 16 findings, 16 confirmed, 10 carrying trades - and all 10 trades identical to the AI's suggestion, 8 of them below the confidence the bulk path is supposed to refuse. Small sample, your test data, not proof of how customers behave. The mechanism is real and it has already been exercised.

Exactly what I change

- **Allocation stops writing `confirmed_at` and `confirmed_by`.** It writes the trade and the due date, which is its own record, and gets its own name in the audit trail (`finding.allocate` rather than a generic update).
- **A material edit clears the approval.** Change the wording, the remedial text, the severity or the grade, and the confirmation comes off - so it has to be given again deliberately.
- Put that rule in **one shared write path**, not in each screen. A comment in `bulk-accept.ts` records that this exact bug was fixed once before; a bug that returns is a missing rule, not a missing fix.

The one place I disagree with the review

Codex wants the contractor's name forced at the moment you allocate. I would not do that. "This is a bricklayer's job" and "Bricklayer means Dave at Wates" change at different rates, and coupling them makes a 100-item pass slower and breaks the day Dave leaves. Keep them separate as data, require both before release, and show the resolved name at allocation as a confirmation rather than another field to fill in.

Changes 3 and 4: the gate, and the empty envelope

3 The publish gate moves onto the server

Publishing is blocked in the browser by `issueBlockers`. The function that actually writes the version and stamps the report as issued, `issueReport` (`report-data.ts:456-489`), checks nothing.

So the gate is a suggestion. A tab left open, or a call sent straight to the database, publishes an unreviewed report - and it looks identical to a properly issued one.

The change: re-check the blockers where the version is written, so the check cannot be skipped. I would put it in the database rather than in a server function, because the database is the only layer the browser cannot route around.

Note This is the one I will not write blind. A gate that is wrong in the wrong direction stops you publishing at all, so it gets tested against a real database before it goes anywhere near production.

4 Nothing external exists for an unpublished report

Trade extracts can be sent before a report is published. `distribute` mints the trade link and sends with no status check at all (`reports.$id.distribute.tsx:98-113`).

Worse than an ungated send, and this is the bit worth reading twice: a client email sent too early **does not get stopped - it loses its link (`email.server.ts:577`) and its attachment (`pdf-attachment.server.ts:82`) and goes out empty**. A client receives an email about a report with nothing in it.

The change: nothing external exists for a report that is not published - full stop. A trade pack for a draft is either refused or carries a loud "DRAFT WORK PACK" identity with none of the client styling. A send that cannot be completed is refused, not half-sent.

Credit where it is due, since it affects the risk count

Two things in the product are already right and should not be touched: confidential findings are excluded from every external channel including share links and trade packs, and the `distributions` table (recipient snapshot, findings, status, opened time, error) is well designed - though it has zero rows, so the audit trail has never actually been exercised.

Changes 5 and 6: the link a supplier can open, and the reason you sent it

5 The close-out email link points somewhere a supplier can open

One line, and it contradicts the file's own comment two hundred lines above it.

`email.server.ts:766` sends the subcontractor to `/reports/<id>` - an in-app address that requires a sign-in they cannot pass. The comment at `:543-545` states plainly that the in-app report URL is never emailed, for exactly that reason.

The change: send the trade token URL - the same page the trade pack uses, which opens with no account. The machinery to build it already exists (`itemListLinkFor`, `:547-571`); the close-out path simply does not call it.

6 The reason for a recipient gets recorded

The distribution plan already works out who should receive a send, and why. It just does not keep the reason.

Findings carry a trade label; the person is resolved from the project directory at send time (`distribution-data.ts:244-271`). Nothing records which rule picked that address.

The change: store the rule alongside the recipient on every send, with the reason. When a client asks "why did my site manager get this", the answer is in the record rather than in someone's memory.

7 The UX tidy-up - deliberately last

Six rows in one Share menu, allocation reachable from three places, four overlapping send actions. Real, and not a launch risk.

The target design already exists inside the product: manual photographic reports are PDF-only and never produce a review page. That is the "different document for a different audience" pattern the review recommends, already shipped and working. The tidy-up is making the rest of the product match it.

What I am deliberately not changing

- **Capture, analysis, triage, issuing and the PDFs.** These are sound. The core product is not the problem.
- **The confirmation workflow itself.** It stays. Only the thing that quietly counts as a confirmation goes.
- **Downloading your own draft PDF.** Reading your own draft is the workflow, not the defect - it just has to keep calling itself a draft, which it now does.

SECTION 3

What needs the database, and how we know it works

Database changes

Change	What it is	Risk if wrong
Share version pinning	One column added to `report_shares`. Existing rows are untouched and keep working.	Low - a link falls back to current behaviour.
Publish gate enforcement	A check where the version is written, so the gate cannot be bypassed.	**High if wrong** - a bad check blocks publishing entirely. This one gets tested against a real database before it goes near production.
Recipient reason	One column, filled on send.	Low - additive.

> **The line I will not cross without a second yes**
>
> I can write and test these changes on a branch. **Running a database migration against your live project is a different decision** and I will not do it on my own initiative. You get told before it happens, and you can read it first.

The tests that have to pass before any of this is released

- Publish v1, edit the wording, publish v2 - **the v1 link still shows v1**, and the changed sentence is absent from it.
- Reopen a published report - the link already sent **still shows the frozen version**, not the working copy.
- A draft report - minting a link is refused, and a trade pack is either refused or clearly marked as a draft.
- Allocate a trade - the confirmation is **unchanged**. Then edit the wording - the confirmation is **cleared**, and Publish is **blocked**.
- Apply bulk suggestions with five below the confidence threshold - **none of them end up confirmed**, and the release check lists them.
- Call the publish path directly, bypassing the screen, with blockers present - **it must fail**.
- A confidential item - absent from the share link, the trade pack and every email.
- The close-out email link - opens with **no sign-in**.
- The one that matters end to end: 47 findings, mixed confidence, one confidential, one unassessable. Triage, allocate, the release check lists exactly 3 blockers, fix them, publish, the client link is frozen, the trade pack is live, edit after publishing - the trade pack shows a change note and **the client link does not move**.

PROPOSAL - NOTHING HERE IS BUILT, COMMITTED OR APPLIED

What I need from you

One

A go. Say go and I will make changes 1, 2, 4 and 5 on a branch, with tests, and show you the diffs. Nothing pushed, nothing applied.

Two

Do the three external surfaces stay switched off until this lands? That is public share links, trade packs, and client email. My advice is yes - those are the three places the product currently makes a promise it cannot keep. Your call, because it is your launch.

Three

A second yes, later, for the database. Changes 3 and 6 add a column and a check to your live Supabase project. I will write and test them, then ask.

What I am not asking for

- No password. No credential of any kind, ever.
- No decision on the UX tidy-up - it is last and it can wait.
- No rush. Nothing here gets worse by waiting a day, and it all gets worse by shipping unfixed.

> **My recommendation, in one line**
>
> Conditional go. No data-loss, no money, no safety risk anywhere in this list - the exposure is evidential and reputational. Ship the capture, analysis, triage, allocation and PDFs as they are; hold the three external surfaces until changes 1 and 2 are in.

Source references were read at commit `1d47100` (tip of `main`) on 11 October 2026. Working copy: branch `p0/report-integrity`, created but empty - no edits made. Findings originally raised by Codex; independently re-verified, and where I disagree it is stated as disagreement.